UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Object-oriented modeling – IV

Readings: ( :chapt. 12)

1

Problem 6: Invoices (Horstmann, 2018, chap. 12)

Create and print out an invoice describing the charges for a set of products in certain quantities.

The invoice should include the billing address, all line items, and total amount due. A line item includes a description, unit price, quantity ordered, and total price.

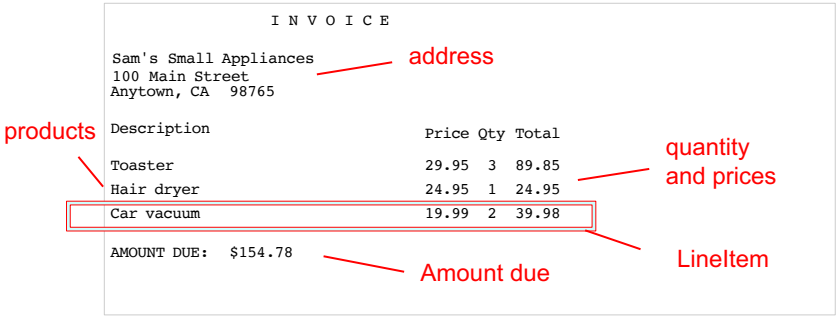

© Scott Cramer/iStockphoto.

For simplicity, do not provide a user interface, and omit complexities such as dates, taxes, and invoice and customer numbers.

7-2

2

Problem 6: Sample invoice (Horstmann, 2018, chap. 12)



7-3

3

Evaluation of class candidates

- Candidates
  - Invoice
  - Address
  - LineItem // Records the product and the quantity
  - Product
  - Description // Attrib. of the Product class
  - Price // Attrib. of the Product class
  - Quantity // Attrib. of LineItem; Not an attribute of Product
  - Total // Computed — not stored anywhere
  - Amount Due // Computed — not stored anywhere
- Classes
  - Invoice
  - Address
  - LineItem
  - Product

7-4

4

Initial UML class diagram (Horstmann, 2018, chap. 12)

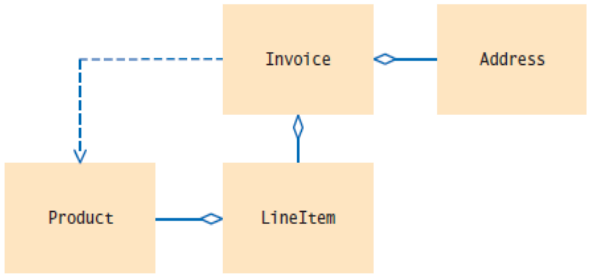


Figure 7 The Relationships Between the Invoice Classes

- The remaining steps of coding and documentation are left as exercice

7-5

5

Object Diagrams

- Several ways of representing objects in UML:
  - write only the class name preceded by a colon and underlined

: Student

  - write the name of specific object with it's class

oneStudent : Student

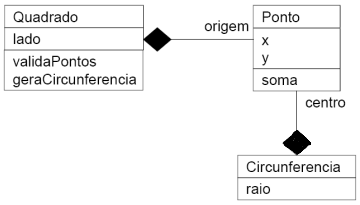
  - multiple objects :

: Student

7-6

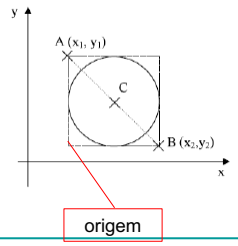
6

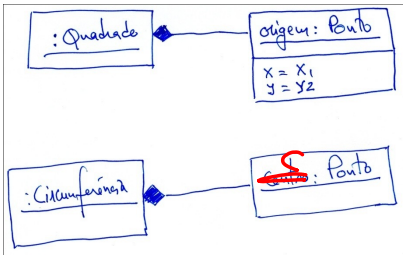
Squares and Circles: example of object diagram



(A possible class diagram)

The object diagram for the given problem instance



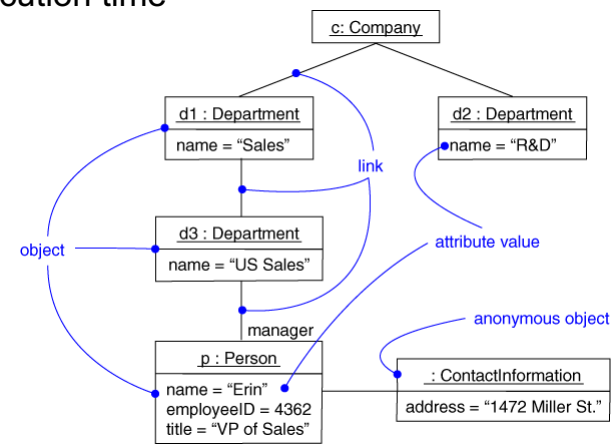


7-7

7

Object Diagram

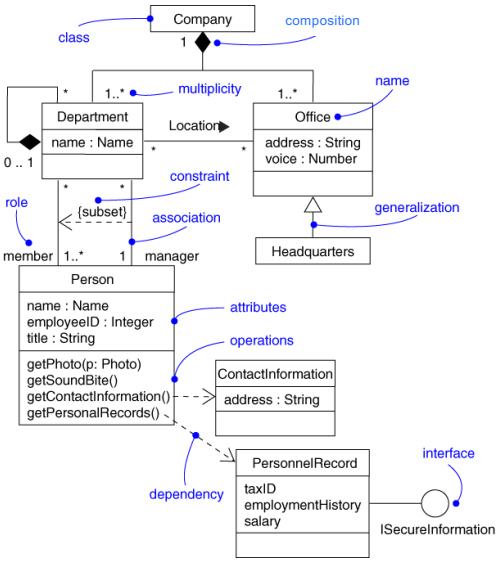
- Captures instances and associations at a given point in execution time



7-8

8

Class Diagram

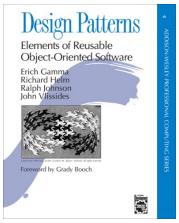


7-9

9

Patterns: a first notion

Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Pattern – Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995



10

Squares and Circles: What class should be responsible for a given task?

(...)

Q3: What should be the class responsible for computing the center and radius of the circunference, given a square?

Re: Two hypothesis:

- Square
- Circunference

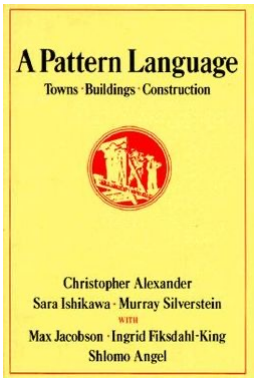
PREVIOUSLY ON OOP

In general, we decide in favour of the class that has all the required information to solve the task. In this case, we decide for Square.

7-11

11

Pattern



"Each pattern describes a ***problem*** which occurs over and over again in our environment, and then describes the core of the ***solution*** to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice", (Christopher Alexander *et al*, *A Pattern Language: Towns, Buildings, Construction*, 1977)

7-12

12

Pattern

"Pattern is a **named** and well-known problem/solution pair that can be applied in **new contexts**, with advice on how to apply it in novel situations and discussion of its **trade-offs**, **implementations**, **variations**, and so forth." (Craig Larman, *Applying UML and Patterns*, 1995)

7-13

13

The Expert pattern

Name:	Expert
Problem:	What is a basic principle by which to assign responsibilities to objects?
Solution: (advice)	Assign a responsibility to the class that has the information needed to fulfill it.

7-14

14

Padrão Expert, discussão

- O Expert leva-nos habitualmente a programas onde o objeto no programa tem o comportamento que é normalmente encontrado no objeto do mundo real.
- O Expert promove o encapsulamento, uma vez que o objeto usa os seus próprios dados para realizar as suas tarefas.

7-15

15

Design patterns

- A design pattern captures *design expertise* abstracted from *existing* design examples
- Design patterns allow to *reuse* design expertise
- Design patterns allow one to study how experts do design

7-16

16

Where are we in OOM?

- Modeling and UML, an Introduction
- Classes and relationship between classes: association, composition, and agregation
- Introduction to UML class and object diagrams
- Examples: i) Squares and Circles; ii) Invoices

- Notion of *design pattern*
- The Expert pattern

17

UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

An overview of Test-driven development

18

Alguns tipos de erros durante a execução de um programa

- falhas de hardware, e.g., o disco falhou
- erros provenientes do sistema operativo, e.g., *drive not ready*
- erros lógicos resultantes da violação de contratos, pelos clientes das classes, e.g., divisão por zero
- erros lógicos, resultantes de violações de invariantes na implementação das classes, e.g., A classe Retangulo processa quatro pontos admitindo que definem um retângulo, quando de facto, assim não é.

19

Erros lógicos depuram-se (quando detetados)!

- Erros lógicos corrigem-se
- …quando detectados
- Testes só mostra a existência de erros; não provam a sua ausência
- Os testes unitários e o JUnit apoiam a depuração

20

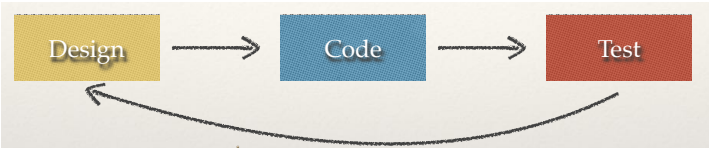
Disadvantages of testing after coding

- Testing does not give direct feedback to design and programming
- Often, after coding, unit testing is omitted
- Often there are not have enough “testable” code segments to write the tests

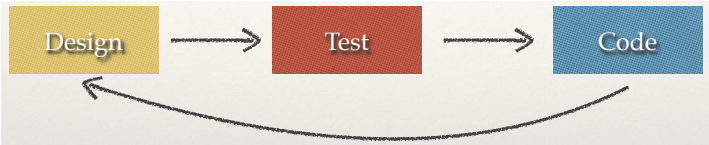
7-21

21

Old school approach



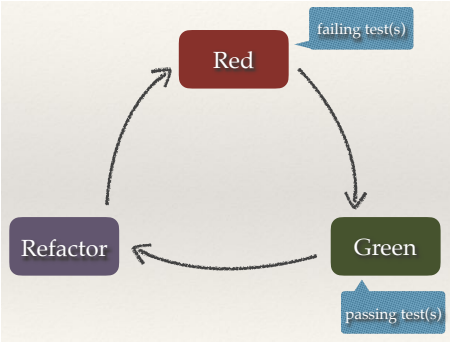
New school approach



7-22

22

The concept

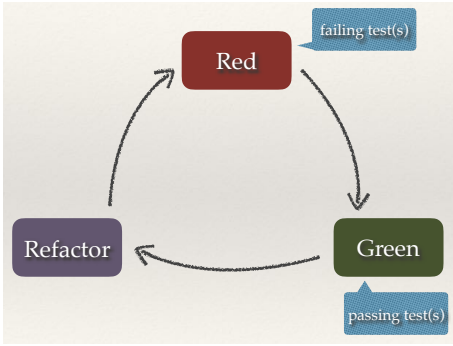


- Test-Driven Development (TDD) is a technique for building software that guides software development by writing tests first

7-23

23

TDD Cycle



1. Write an automated test case
2. Produces the *minimum* code to pass test
3. Refactor the code to acceptable standards

7-24

24

Refactoring, in short

- Change the internal structure of the code without changing it's functionality
- Similar to arithmetic factorization:

$$ab + ac = a(b+c)$$

7-25

25

Benefits of TDD, in brief

- Software is developed in short iterations
- Validate the production code
- Protect the production code for each change
- Resolve any failure rapidly

7-26

26

Benefits of TDD

Better Design	Cleaner code (because of refactoring)
Safer refactoring	Increased quality
Better code coverage	Tests serve as documentation
Faster debugging	Most often, the failing code/test is in the most recently changed code
Self-documenting tests	Test-cases show/indicate how to use the code

7-27

27

Unit test characteristics

- Very limited in scope
- Independent from each other
- Run each time something is changed
- Run automatically
- Run fast; the fastest the better

7-28

28

JUnit

- Tests are expressed in ordinary source code
- The execution of each test is centered on an instance of a TestCase object
- Groups of tests can be collected together, and their results of running them all will be reported collectively

29

Junit, some assert methods

assertEquals()/ assertNotEquals()	Invokes the equals() methods on the arguments to check whether they are equal
assertSame()/ assertNotSame()	Uses == on the arguments to check whether they are equal
assertTrue()/ assertFalse()	Checks if the given boolean argument evaluates to true or false
assertNull()/ assertNotNull()	Checks if the given argument is null or NOT null
assertArrayEquals()	Checks if the given array arguments passed have same elements in the same order

30

JUnit – best practices

- Test name should reflect the intent/purpose & describe the feature/specification
- Unit test should test a *single* program element
- Readability matters in the tests - use comments as appropriate
- Tests obvious functionality as well as corner cases - both are important

31

TDD in practice

1. Add a Test
2. Run all tests and see the new one fail
3. Write some code
   - Only code that is designed to pass the test
   - No additional functionality should be included because it will be untested

32

TDD in practice

4. Run the automated tests and see them succeed
   - If tests pass, programmer can be confident code meets all tested requirements
5. Refactor code
   - Cleanup the code
   - Rerun tests to ensure cleanup did not break anything

Repeat

7-33

33

TDD, some limitations

- Counter-intuitive and hard to learn
- Difficult in some situations
  - GUIs, Relational Databases, Web Service
  - Requires mock objects

7-34

34

To probe further

- Beck, K. *Test-Driven Development by Example*, Addison Wesley, 2002

7-35

35

Check point: what's wrong?

```
public class Triangle {
    private Point pa, pb, pc ;
    public Triangle (Point a, Point b, Point c) {
            pa = a;                 pb = b;                 pc = c;
    }
    public void setPA(double x, double y) { pa = new Point(x,y);}
    public void setPB(double x, double y) {
            assert pa.getX() != x || pa.getY() != y; pb = new Point(x,y);
    }
    public void setPC(double x, double y) {
            assert pb.getX() != x || pb.getY() != y; pc = new Point(x,y);
    }
    // ...
}
```

A triangle cannot be defined by 3 collinear points nor by 3 equal poins

7-36

36

Check point: what's wrong?

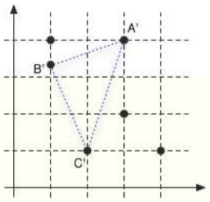
```
public class Point {
    private int x, y;
    public Point(int x, int y) {
            setX(x);
            setY(y);
     }
     public int getX() {  return x; }
     public double dist(Point P) {
            double dx = x - P.getX();
            double dy = y - P.getY();
            return Math.sqrt(dx*dx + dy*dy);
      }
      public boolean isCollinear(Point Q, Point R) {
            return (Q.getX() - x)*(R.getY() - y) == (R.getX() - x)*(Q.getY() - y);
      }
}
```

Collinearity is NOT a responsability of Point

7-37

37

Check point: a solution




```
public class Triangle {
    private Point pa, pb, pc ;
    public Triangle (Point a, Point b, Point c) {

            if(new Reta(a,b).isCollinear(c) {
               throw new IllegalArgumentException("A triangle cannot be
                                                   defined by 3 collinear points");

            pa = a;
            pb = b;
            pc = c;
     }
// ...
}
```

If 3 points are collinear
Triangle constructor throws an Exception

7-38

38